

AWS Certified Generative AI Developer — Professional

The Use Case

- Global manufacturer — 50+ production facilities, 15 countries, thousands of components, hundreds of suppliers
- Requirements:** automate complex supply chain decisions, optimise inventory without disrupting production, proactively detect disruptions, coordinate suppliers/logistics/plants, **transparent reasoning**, human review of critical decisions, integration with existing enterprise systems
- Results:** CoT reasoning improved decision quality 37%; dynamic model selection cut response time 42%

Domain 1 vs Domain 2

Domain 1 asked **what to build**. Domain 2 asks **how to build it**.

Throughout Domain 1 the guidance was "Agents are non-deterministic — prefer Flows in regulated settings." **Task 2.1 is the other side of that argument:** when autonomy genuinely is needed, how do you build it and how do you make it safe?

Transcript error worth knowing: it mentions "DALL-E for image creation" in a Bedrock context. DALL-E is OpenAI's and is **not available on Bedrock**. Bedrock image models include Stability AI, Amazon Titan Image Generator and Nova Canvas. If DALL-E appears in an exam option, it is a distractor.

1. Autonomous Systems — Agents and Memory

What an agent is

```
Reason -> "I need inventory data"
Act    -> call the inventory API
Observe -> "stock is low, check suppliers next"
Reason -> choose the next tool ...
```

You don't define the path — you only supply the tools.

Bedrock Agents — three parts

Component	Role
Foundation model	The reasoning engine
Instructions	Role and scope: "you are the inventory optimisation agent..."
Action groups	OpenAPI schema + Lambda — the tools it can call

Optionally a **Knowledge Base** (RAG) and **Guardrails**.

Single vs multi-agent — the first decision

Choose	When
Single agent	One domain, a handful of tools, sequential work
Multi-agent	Distinct expertise required, work can run in parallel, each part is complex in its own right

Why multi-agent here: demand forecasting is statistics, supplier management is negotiation, logistics is routing. Teaching one agent all three makes its instructions so large it becomes confused. Each agent gets its own focus, tools and prompt.

```
Demand forecasting agent
Inventory optimisation agent
Supplier management agent
Logistics coordination agent
```

Bedrock Agents vs Strands vs Agent Squad

Service	Role
Amazon Bedrock Agents	Managed service — build, test and deploy a single agent; AWS handles the loop
AWS Strands Agents	Code-first framework — specialised agents with custom action schemas and execution handlers
AWS Agent Squad	Orchestration across agents — task delegation, communication protocols, combining results

Memory hierarchy — three tiers

Operational	-> current session, what tools ran	-> ElastiCache
Tactical	-> recent decisions, recent issues	-> DynamoDB
Strategic	-> long-term patterns and learning	-> DynamoDB / S3

Memory type	Contains	Stored in
Short-term	Conversation context, current session state	ElastiCache (sub-ms) / DynamoDB
Long-term	Learned patterns, persistent knowledge	DynamoDB / S3 / Knowledge Base

The DynamoDB conversation history from Task 1.6 *is* the short-term layer; this scenario adds two tiers above it.

MCP between agents

Task 1.5 introduced MCP as a model-to-tool protocol. **Here it also carries agent-to-agent communication** — each agent keeps its specialised focus while sharing critical insights, without writing a custom integration for every agent pair.

2. Advanced Problem-Solving

ReAct — Reason + Act

```
REASON  -> "component shortage; check production impact first"
ACT      -> query production system
OBSERVE -> "3 lines affected, 2-day delay"
REASON  -> "now look at alternatives"
ACT      -> query supplier database ...
```

The model doesn't need a complete plan upfront — it takes one step, sees the result, then decides the next.

Transcript's structured reasoning for a component shortage

- 1. Reason about production impact
- 2. Query inventory systems
- 3. Analyse alternatives
- 4. Evaluate cost implications
- 5. Generate recommendations

Why Step Functions ReAct, when agents already do ReAct

Who controls the loop	
Agent's built-in ReAct	The model — any number of steps, any order
Step Functions ReAct	You — reasoning steps are defined; the model still reasons within each step

Step Functions adds **stopping conditions, state persistence, visibility and auditability**. It is the middle ground between fully fixed Flows and fully free Agents.

Chain-of-Thought as a workflow

In Task 1.6, CoT was a *prompting* technique inside one call. Here it becomes a workflow: each reasoning step is a separate call whose **intermediate result persists in state**, feeding the next step and remaining inspectable.

Transcript: 37% improvement in decision quality versus a single-step approach.

State persistence and iterative refinement

- Each step builds on the previous step's insight — the solution is progressively refined
- If step 4 fails, results from steps 1-3 survive; the work isn't repeated

Problem decomposition — Lambda

```
"Supplier X is two weeks late – what do we do?"
| Lambda decomposes
1. Which production lines are affected?
2. Are alternate suppliers available?
3. How much buffer inventory exists?
4. What is the impact on customer commitments?
```

Sub-problems route to different specialised agents. **Dynamic decomposition** means the split depends on the problem's characteristics — a weather disruption decomposes differently from a quality issue. Same pattern as query decomposition in Task 1.5.

Evaluating reasoning quality

Check	Looks for
Consistency	Do the steps contradict each other?
Logical validity	Does the conclusion follow from the premises?
Outcome validation	Is the recommendation actually feasible?

A model can produce confident reasoning that is internally inconsistent — step 2 says stock is 500, step 4 assumes 5,000. **Only reading both steps together reveals it.**

3. Safeguarded AI Workflows

The most important section in Task 2.1. An agent is autonomous — it decides and acts on its own. Nothing stops it if it goes wrong. Agent-specific risks: infinite reasoning loops, unbounded tool calls (cost), accessing resources it shouldn't, and cascading failures.

Stopping conditions — Step Functions

```
Max iterations:      10
Max execution time:  5 minutes
Termination criteria: confidence > 0.8, or answer found
```

A ReAct loop cannot stop itself — the model will keep deciding it needs "a bit more data". Step Functions provides state transition rules and timeout configuration. **Stopping the agent is your responsibility, not the agent's.**

Timeouts — Lambda and workflow

- **Lambda-level** timeout (max 15 min) and **Step Functions** timeouts on states and the whole execution
- **Graceful termination:** save partial work, return what was produced, shut down cleanly

IAM least privilege

If the inventory-query Lambda holds `dynamodb:*`, the agent can **delete** data — even though you never defined a delete action. The action group is not the security boundary; **IAM is.**

```
inventory-query-lambda
-> dynamodb:GetItem, dynamodb:Query
-> on inventory-table only
-> nothing else
```

Don't hand the agent the whole keyring — give it the one key for the one room.

Circuit breakers — Step Functions + CloudWatch alarms

- Error rate crosses a threshold → circuit opens → processing halts
- **Cascading failure:** agent A fails, agent B was waiting on it, agent C on B — one problem takes down the system. The circuit breaker cuts the chain.
- **Transcript's real example:** during a major weather event that disrupted suppliers, the system **automatically switched to conservative planning mode** until conditions normalised — degraded, not stopped. Graceful degradation (Task 1.2) applied to agents.

Input validation

Tool-call parameters are generated by the model, so they can be wrong: `delete_inventory(quantity: -500)`. Lambda must validate structure, ranges and business rules **before executing** — the agent's output reaches real systems.

The five layers

```
Agent wants to act
-> [IAM]           is it even permitted?
-> [Input validation] are the parameters sane?
-> [Timeout]       how long may it take?
-> [Stopping]      how many loops may it run?
-> [Circuit breaker] is the system healthy?
```

Each layer stops something different; none substitutes for another.

4. Model Coordination

Task-based routing

Claude 3 Sonnet	-> complex reasoning
Titan Text	-> report generation
Titan Embeddings	-> similarity matching

Reasoning is hard and expensive — use the strongest model. Report generation is simple — a cheaper model suffices. Note that **Titan Embeddings isn't a "lesser" model**; embedding and text generation are different jobs entirely.

Dynamic model selection

Factor	Influence
Performance	How has this model done on this task historically?
Cost	Remaining budget
Latency	How fast is the answer needed?
Current load	Is the model being throttled?

Transcript: 42% reduction in response time, because not every request goes to the largest model. Same idea as AppConfig in Task 1.2, but evaluated at runtime.

Ensembles — aggregation techniques

Technique	How and when
Majority voting	Three models answer; the most common answer wins. For classification and discrete choices.
Weighted averaging	Each model's output weighted by its historical performance . For numeric predictions — demand forecasts, cost estimates. This is what the transcript uses.

- Custom aggregation logic lives in **Lambda**
- Weights are not fixed — better-performing models earn more influence over time
- **Cost**: N model calls per request. Reserve for decisions where accuracy is critical.
- Secondary benefit (from Task 1.2): **disagreement between models is itself a signal** to route to human review

Fallback mechanisms

primary model FAILS	-> alternate model
primary model LOW CONFIDENCE	-> alternate model

The second trigger matters — no error was raised, the model simply isn't sure. Degradation chain from Task 1.2: strong model → smaller model → rule-based → human.

Multi-model pipelines in Step Functions

Stage 1: Titan Embeddings	-> find similar past cases
Stage 2: Claude	-> reason about the situation
Stage 3: Titan Text	-> generate the report

The four coordination patterns

Pattern	Does	Cost
Task routing	Model chosen by task type	Low (1 call)
Dynamic selection	Model chosen by runtime factors	Low (1 call)
Ensemble	Several models, outputs combined	High (N calls)
Fallback	Switch on failure or low confidence	Low (usually 1)

5. Collaborative AI Systems

Escalation criteria — where the line sits

confidence > 0.9	-> automatic
0.7 - 0.9	-> AI acts, human reviews afterwards
< 0.7	-> human approves first

Risk-based triggers (transcript): major production schedule adjustments, strategic inventory positioning, supplier relationship changes, responses to major disruptions. Routine reordering stays automated; changing a supplier is a human decision.

Rule: high risk OR low confidence → human. Use both factors, not confidence alone.

Step Functions human approval

```
workflow runs
-> human approval step
-> WAIT (task token issued, execution pauses)
-> human approves / rejects
-> token returned -> workflow resumes
```

- The workflow **pauses, it does not terminate** — state survives, so approval two hours later resumes from the same point
- Timeouts still apply — no response in 24 hours triggers escalation or auto-reject
- Same deliberate human gate as clinical validation (Task 1.2) and three-tier prompt approval (Task 1.6)

Human augmentation — the key idea

```
AI produces:
  situation analysis
  options with trade-offs
  recommendation with reasoning
  supporting data
  |
Human: read -> adjust -> approve
```

The point is not to replace the human, but to compress their work. A two-hour review becomes ten minutes; the decision stays theirs, but the research is done. Transcript: "comprehensive analysis packages for human reviewers."

Feedback collection and storage

- **API Gateway** endpoints capture structured feedback with authentication
- Capture: approve/reject, the reason, and — most valuable — **what the human changed**. A correction contains both the error and the right answer.
- **DynamoDB with GSIs** so patterns are queryable: which decision types get rejected most, which agent's confidence is least reliable. Without indexes this means full table scans.

```
PK: decision_id    SK: timestamp
    reviewer, action, reason, original, corrected
GSI: decision_type + timestamp
GSI: reviewer + timestamp
```

6. Intelligent Tool Integrations

Action group anatomy

Action Group	
+-- OpenAPI schema	(tells the agent what the tool does)
+-- Lambda function	(does the work)
+-- IAM permissions	(what the Lambda may touch)

The OpenAPI schema is the agent's instruction manual

The `description` fields matter more than anything else — the agent reads them to decide which tool to call and when. A technically valid schema with vague descriptions produces an agent that picks the wrong tool, and the model gets blamed for a documentation problem. **The OpenAPI schema is your prompt to the model.**

Parameter validation — two layers

Layer	Checks
Bedrock Agents	Schema conformance — types, required fields
Lambda	Business rules — does this facility exist, is the quantity positive, is this combination valid, is this user authorised

Bedrock's validation is not sufficient — it checks types, not meaning.

Error handling — errors are read by the model

Poor:	"Error 404"
Good:	"Facility PLANT-99 not found. Valid facilities are PLANT-01 through PLANT-50. Retry with a valid facility ID."

The second gives the agent a chance to correct itself. **Write error messages knowing a model will read them.**

Error class	Agent should
Retryable	Try again (throttling, timeout)
Fixable	Correct the parameters and retry
Fatal	Stop and escalate

The error response should indicate which class it is.

AWS Strands API — custom behaviour

Bedrock Agents is managed and therefore constrained. Strands lets you define **action schemas** and **execution handlers** yourself. Transcript's example: **multi-echelon inventory optimisation** — a complex algorithm, not a simple API call.

Monitoring tool usage

- Track invocation counts, error rate per tool, latency per tool, and which tool sequences the agent favours
- A tool with a 40% error rate usually has a bad description** — the agent is misunderstanding it
- A tool that is never called** may be invisible to the agent
- Tool usage patterns reveal how the agent is reasoning

Enterprise integrations in this scenario

ERP systems	-> inventory and production data
Supplier platforms	-> supplier information
Logistics systems	-> shipping status
External APIs	-> disruption monitoring

Each is an action group with its own OpenAPI schema and Lambda.

7. Model Extension Frameworks — MCP Servers

MCP appears for the third time, from a new angle each time: Task 1.5 (what the protocol is), earlier in 2.1 (agent-to-agent), and now **building MCP servers**.

Action Groups vs MCP

Action Groups		MCP
Consumer	Bedrock Agents	Any MCP-compatible client
Definition	OpenAPI schema	MCP protocol
Reuse	Within that agent	Across models and platforms

Where to host an MCP server

Host	Suited to	Why
Lambda	Calculations, data retrieval, simple transformations, web search, optimisation solvers, forecasting, geospatial analysis	Serverless, pay per call, instant scaling. Stateless, 15-min limit, modest memory.
Amazon ECS	Simulation engines, advanced optimisers, code execution, image processing	Long-running, high memory/CPU/GPU, specialised environments with custom runtimes and dependencies, can be stateful

small + stateless + under 15 min -> Lambda
heavy + special environment + long -> ECS

Lambda is a calculator — take it out, use it, put it away. ECS is a workshop — always open, all the equipment on hand.

API Gateway as an MCP endpoint

Expose **existing REST services** through an MCP-compatible interface. The underlying service doesn't change; API Gateway translates. Same adapter principle as Task 1.2.

MCP client libraries and tool discovery

- **Client libraries** hide the protocol's complexity and give consistent access patterns across different models and tools
- **Tool discovery:** rather than hardcoding a tool list, the agent **asks what is available**. Add a new tool and the agent finds it — no code or prompt change.
- **Trade-off:** highly extensible, but less predictable — the agent's capabilities change without a deployment, which can be a problem in regulated environments

Foundation Model / Agent
MCP client library
MCP protocol
Lambda / ECS / API Gateway
(light) (heavy) (existing APIs)

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.